



Direct-to-Ids loads in Triton for GFX9

Alexander Weinrauch
7th June 2025

Content

- What are direct-to-lds loads
- ISA overview of direct-to-lds loads
- Hardware limitations on GFX9
- Representation in TTGIR
- Pipelining with direct-to-lds loads
- Synchronization
- Impact on memory layouts

Global Load

vs

Global load to lds

- Transfers the data from HBM to the **register**
- Reads global address from **VGPR** (per lane)
- Takes a **VGPR** to store into
- Increases **vmcnt** during issue
- Decreases **vmcnt** if the data is stored in the register
- Ordering between loads is guaranteed

- Transfers the data from HBM to **LDS**
- Reads global address from **VGPR** (per lane)
- Takes the LDS address in a **SGPR** (per warp)
- Increases **vmcnt** during issue
- Decreases **vmcnt** if the complete data is stored in LDS
- Ordering between loads is guaranteed
- Order with **ds_*** ops is not guaranteed
 - Use **lgkcnt**

ISA

- `global_load_lds_dword v[184:185] off`
 - `dword` => 32bit per lane
 - Supports 8, 16, 32, 96 (gfx950), 128 (gfx950)
 - `v[184:185]`, v184 global ptr, v185 offset to global ptr
 - `off` -> v185 is used as an offset
 - Also supports constant offset but it's applied to the global and lds address
- Where is the LDS location?
 - Stored in `m0` a special **SGPR**
- LDS addr calculation:
 - For size <= 32bit: $\text{LDS_ADDR} = \text{m0} + \text{inst_offset} + (\text{TIDInWave} * 4)$
 - For size > 32bit: $\text{LDS_ADDR} = \text{m0} + \text{inst_offset} + (\text{TIDInWave} * 16)$
- We can only use **DWORD** and **DWORDx4** because other sizes upcast before storing to LDS

Buffer load to lds

- `buffer_load_dwordx4 v8, s[16:19], 0, lds`
 - `v8` holds the index into the buffer
 - `s[16:19]` buffer descriptor
 - 0 constant offset (offen)
 - `lds` => stores to lds
- See Guiseppe's presentation about buffer ops for details on the buffer descriptor and the offset

HW Limitations of direct-to-lds

- Only **gather** into LDS no scattering -> how do we do swizzling?
- Gfx942 only has **DWORD** as largest load width
- No **DWORD_X2**
- **< 32bit** not useful for Triton because they extend the data to 32bit
- No separate hardware counter for direct-to-lds

Representation in TTGIR

- We reuse the upstream primitives to ensure combability and piggyback a bit on existing functionality.
- `ttg.async_copy_global_to_local %ptrs %memDesc %mask %other -> ttg.async.token`
 - Emits the direct-to-lds load
 - AsyncToken is used for synchronization
- `ttg.async_commit_group [ttg.async.token...] -> ttg.async.token`
 - We do not have that concept, so they are discarded during lowering
 - We still emit them to be compatible with upstream functionality
- `ttg.async_wait [ttg.async.token...] -> ttg.async.token`
 - Emits **gpu.barrier** in Membar analysis
 - LLVM ~~should~~ emit the correct waitcnt vmcnt for us
 - [SWDEV-52112][SWDEV-514726] requires us to do it (more later)
- `ttg.local_load %memDesc token %asyncToken`
 - Local loads can consume AsyncToken to model the dependency

How do we pipeline

- Stream pipeliner replaces **tt.load** with:
 - 3 dimensional **local_alloc** outside of the loop:
 - First dim is the buffer index to double, triple.. buffer
 - Second and third dim are based on the load dimension
 - Compute buffer index based on loop iteration and creates **subview** of allocation
 - **tt.load** replaced with **tt.load** -> **ttg.local_store** -> **ttg.local_load**

IR example for pipelined loop after combining

```
%66:6 = scf.for %arg10 = %c0_i32 to %65 step %c1_i32 iter_args(%arg11 = %cst_1, %arg12 = %42, %arg13 = %58, %arg14 = %c0_i32, %arg15 = %63, %arg16 = %64) {  
  %87 = tt.addptr %arg12, %cst  
  %88 = tt.addptr %arg13, %cst_0  
  %89 = tt.load %87  
  %90 = ttg.local_load %arg15  
  %91 = ttg.local_load %arg16  
  %92 = tt.load %88  
  %93 = tt.dot %90, %91, %arg11  
  %94 = arith.addi %arg14, %c1_i32 : i32  
  %95 = arith.cmpi slt, %94, %c1_i32 : i32  
  %96 = arith.select %95, %94, %c0_i32 : i32  
  %97 = ttg.memdesc_subview %61[%96, %c0_i32, %c0_i32]  
  ttg.local_store %89, %97  
  %98 = ttg.memdesc_subview %62[%96, %c0_i32, %c0_i32]  
  ttg.local_store %92, %98  
  scf.yield %93, %87, %88, %96, %97, %98  
}
```

How do we pipeline

- Stream pipeliner replaces **tt.load** with:
 - 3 dimensional **local_alloc** outside of the loop:
 - First dim is the buffer index to double, triple.. buffer
 - Second and third dim are based on the load dimension
 - Compute buffer index based on loop iteration and creates **subview** of allocation
 - **tt.load** replaced with **tt.load** -> **ttg.local_store** -> **ttg.local_load**
- With **TRITON_HIP_USE_ASYNC_COPY=1** we replace the last part with
 - **ttg.async_copy_global_to_local** -> **ttg.async_commit_group** -> **ttg.async_wait** -> **ttg.local_load**
- Pipelining with **AsyncCopy** will use one LDS buffer more
 - Because we are not using a register buffer
- After scheduling the loop we will combine AsyncWaits
- Very similar to upstream pipeliner, in fact the nvidia pipeliner works OOB now

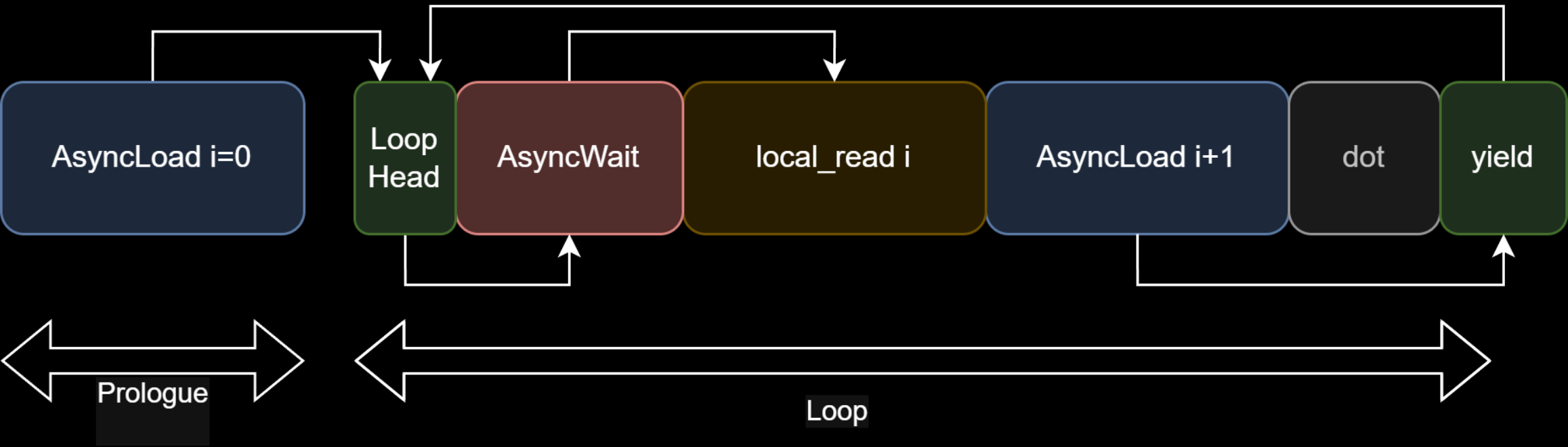
IR example for pipelined loop before combining

```
%68:8 = scf.for %arg10 = %c0_i32 to %67 step %c1_i32 iter_args(%arg11 = %cst_1, %arg12 = %42, %arg13 = %53, %arg14 = %c0_i32, %arg15 = %62, %arg16 = %66, %arg17 = %59, %arg18 = %63) {
  %90 = ttg.async_wait %arg15, %arg16 {num = 0 : i32}
  %91 = tt.addptr %arg12, %cst
  %92 = tt.addptr %arg13, %cst_0
  %93 = arith.addi %arg14, %c1_32
  %94 = arith.cmpi slt, %93, %c2_i32
  %95 = arith.select %94, %93, %c0_i32
  %96 = ttg.memdesc_subview %56[%95, %c0_i32, %c0_i32]
  %97 = ttg.async_copy_global_to_local %91, %96
  %98 = ttg.async_commit_group %97
  %99 = ttg.local_load %arg17 token %98
  %100 = ttg.memdesc_subview %57[%95, %c0_i32, %c0_i32]
  %101 = ttg.async_copy_global_to_local %92, %100
  %102 = ttg.async_commit_group %101
  %103 = ttg.local_load %arg18 token %90
  %104 = tt.dot %99, %103, %arg11
  scf.yield %104, %91, %92, %95, %98, %102, %96, %100
}
```

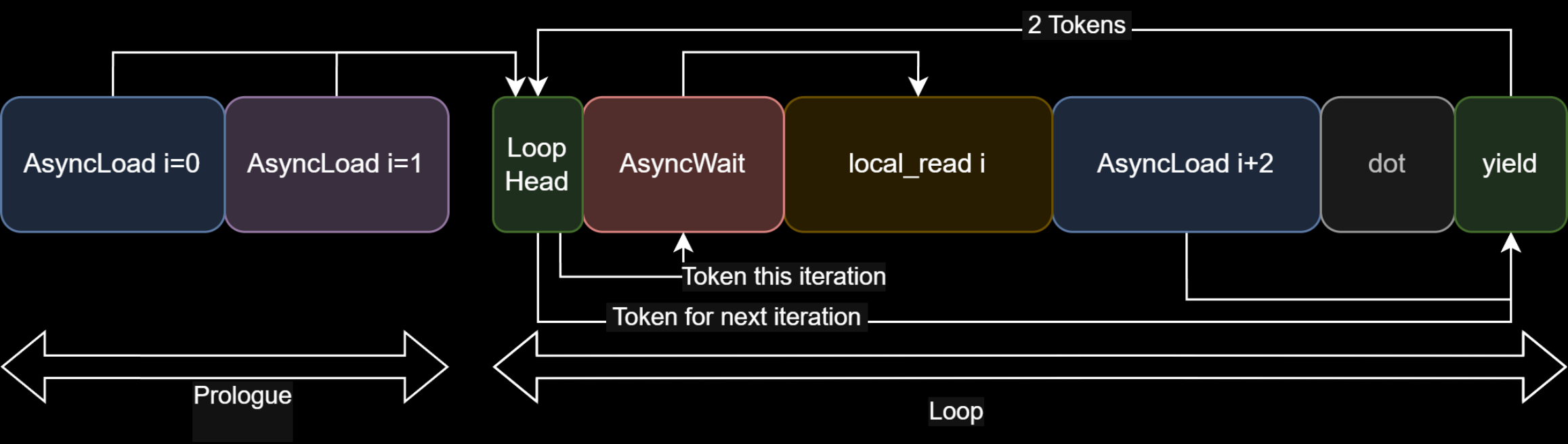
Interleaving Loads for deeper pipelines

- What **vmcnt(x)** do we expect for the **AsyncWait**?
 - $\text{num_stages} == 2 \Rightarrow \text{AsyncCopy result is used in next iteration} \Rightarrow \text{vmcnt}(0)$
 - $\text{num_stages} == 3 \rightarrow \text{AsyncCopy result used from 2 iterations ago} \Rightarrow \text{vmcnt}(x) \text{ where } x = \# \text{AsyncCopiesPerIteration}$
 - In general $\Rightarrow \text{vmcnt}(x) \text{ where } x = (\text{num_stages} - 2) * \# \text{AsyncCopiesPerIteration}$
- What to we get?
 - Always **vmcnt(0)** from LLVM ☹
 - LLVM is unable to see dependencies between **ds_reads** and **AsyncCopies**
 - We have the information in form of the **AsyncTokens**

Pipeline AsyncToken flow num_stages==2



Pipeline AsyncToken flow num_stages==3



Determine correct vmcnt

- We should **never** overestimate the vmcnt to always produce correct assembly
- For each AsyncWait traverse the def-chain of each AsyncToken:
 - Along the def-chain we count the **number of interleaved AsyncCopies** (#instructions)
 - Can have multiple paths due to control flow (e.g. epilogue, loop carried, if/else...)
 - Take **minimum** of all paths to ensure correctness
 - Reuse the num attribute from AsyncWait
 - Before UpdateAsyncWaitCount pass num stands for the number of commit groups
 - Afterwards it will represent the vmcnt => We probably should rewrite it to a new OP to make that more clear

Example1

```
// Emits 1 direct to lds instruction
%0 = ttg.async_copy_global_to_local %arg3, %arg1
// Emits 2 direct to lds instructions
%2 = ttg.async_copy_global_to_local %arg4, %arg2

// Do not wait on the second async_copy => waitcnt 2
// CHECK: ttg.async_wait {{.*}} {num = 2}
%9 = ttg.async_wait %0 {num = 0 : i32}
// No async_copies in between => waitcnt 0
// CHECK: ttg.async_wait {{.*}} {num = 0}
%10 = ttg.async_wait %2 {num = 0 : i32}
```


Example2

```

// Emits 1 direct to lds instruction
%0 = ttg.async_copy_global_to_local %arg3, %arg1
// Emits 2 direct to lds instructions
%2 = ttg.async_copy_global_to_local %arg4, %arg2

%4 = ttg.async_copy_global_to_local %arg3, %arg1
%6 = ttg.async_copy_global_to_local %arg4, %arg2
%8:4 = scf.for %arg15 = %0, %arg16 = %4, %arg17 = %2, %arg18 = %6
  // CHECK: ttg.async_wait {{.*}}, {{.*}} {num = 3
  %10 = ttg.async_wait %arg15, %arg17 {num = 2 : i32}
  %11 = ttg.async_copy_global_to_local %arg3, %arg1
  %13 = ttg.async_copy_global_to_local %arg4, %arg2
  scf.yield %arg16, %11, %arg18, %13
}
// CHECK: ttg.async_wait {{.*}}, {{.*}} {num = 0
%9 = ttg.async_wait %8#0, %8#1, %8#2, %8#3 {num = 0 : i32}

```

Dependency between AsyncCopy and ds_reads

- **Membar** only works on the allocation level so it will not see that the subviews in LDS are disjoint
 - This means we get a barrier in front of the local_loads we created to pipeline the loads
- The **LocalWrites** produced by pipelining consume the AsyncToken from AsyncWait
 - Filter dependencies between **AsyncCopy** and **LocalLoad** if **LocalLoad** consumes **AsyncToken** from **AsyncWait**
- LLVM suffers from the same problem
 - We need to add alias information that the **ds_reads** do not alias with the **AsyncCopies**
- This is done by annotating all **ttg.local_loads** before we run membar analysis.
 - If we see an annotated **LocalLoad** during membar analysis we do not emit a gpu.barrier
 - If we see an annotated **LocalLoad** during lowering to LLVM we do set alias groups to tell LLVM it does not alias with **AsyncCopies**
- **AsyncCopies** get “amdgpu.AsyncCopies” as their Alias group
- **LocalLoads** get “amdgpu.AsyncCopies” as their noAlias group
 - Sidenote: they also get “amdgpu.LocalLoads” as alias group otherwise they will sync all other ds_writes

Layout Constraints with AsyncCopy

Blocked Layout:

1. **sizePerThread** * **bitsPerElement** needs to be 32 or 128 (gfx950)
 - Otherwise we would need to write **strided** to LDS which we cannot do

* For readability we always talk about the fastest (order[0]) dimension if not noted otherwise. Ignore swizzling for now

Size Per Thread

- Assume threadsPerWarp = 4, dtype=float32
- sizePerThread=[1,1], threadsPerWarp = [4,1]
 - 

D	1	2	3	4	5	6	7	8
1	1	2	3	4	5	6	7	8
2	9	10	11	12	13	14	15	16
3	17	18	19	20	21	22	23	24
4	25	26	27	28	29	30	31	32
5	33	34	35	36	37	38	39	40
6	41	42	43	44	45	46	47	48
7	49	50	51	52	53	54	55	56
8	57	58	59	60	61	62	63	64

- sizePerThread=[2,1], threadsPerWarp = [4,1]
 - Dwordx2 not supported => splits into 2 loads (red, green)
 - Strided writes 

D	1	2	3	4	5	6	7	8
1	1	2	3	4	5	6	7	8
2	9	10	11	12	13	14	15	16
3	17	18	19	20	21	22	23	24
4	25	26	27	28	29	30	31	32
5	33	34	35	36	37	38	39	40
6	41	42	43	44	45	46	47	48
7	49	50	51	52	53	54	55	56
8	57	58	59	60	61	62	63	64

Layout Constraints with AsyncCopy

Blocked Layout:

1. **sizePerThread** * **bitsPerElement** needs to be 32 or 128 (gfx950)
 - Otherwise, we would need to write **strided** to LDS which we cannot do
2. If **threadsPerWarp** spans multiple rows we need to **exhaust** the fastest dim
 - Otherwise, we would have **gaps** in LDS but we write coalesced
 - We could split it into multiple loads+masking but this is not implemented

* For readability we always talk about the fastest (order[0]) dimension if not noted otherwise. Ignore swizzling for now

ThreadsPerWarp

- Assume threadsPerWarp = 16, dtype=float32
- sizePerThread=[1,1], threadsPerWarp = [8,2]
 - 

D	1	2	3	4	5	6	7	8
1	1	2	3	4	5	6	7	8
2	9	10	11	12	13	14	15	16
3	17	18	19	20	21	22	23	24
4	25	26	27	28	29	30	31	32
5	33	34	35	36	37	38	39	40
6	41	42	43	44	45	46	47	48
7	49	50	51	52	53	54	55	56
8	57	58	59	60	61	62	63	64

- sizePerThread=[1,1], threadsPerWarp = [4,4]
 - Strided writes 

D	1	2	3	4	5	6	7	8
1	1	2	3	4	5	6	7	8
2	9	10	11	12	13	14	15	16
3	17	18	19	20	21	22	23	24
4	25	26	27	28	29	30	31	32
5	33	34	35	36	37	38	39	40
6	41	42	43	44	45	46	47	48
7	49	50	51	52	53	54	55	56
8	57	58	59	60	61	62	63	64

Layout Constraints with AsyncCopy

Blocked Layout:

1. **sizePerThread** * **bitsPerElement** needs to be 32 or 128 (gfx950)
 - Otherwise we would need to write **strided** to LDS which we cannot do
2. If **threadsPerWarp** spans multiple rows we need to exhaust the fastest dim
 - Otherwise we would have **gaps** in LDS but we write coalesced
 - We could split it into multiple loads+masking but this is not implemented
3. If **sizePerThread** is not 1 it needs to be **contiguous** in HBM
 - This would require **splitting** into multiple loads and would write strided

Shared+Blocked Layout:

1. If `shared_order != blocked_order` **sizePerThread** has to be 1
 - Would require splitting and strided writes
- `AMDCoalesceAsyncCopy` adjusts layouts after pipelining to try to satisfy these constraints

* For readability we always talk about the fastest (order[0]) dimension if not noted otherwise. Ignore swizzling for now

Swizzling

- Apply **swizzling** to global pointers since we must write coalesced into LDS
- Solution1 apply reverse of SwizzleSharedEncoding to BlockedLayout
 - Linear layouts allow us to easily apply/compose it to the blocked layout
 - Resulting LL breaks assumptions about distributed layouts
 - E.g. ExpandDim is no simple op anymore because it would require warp shuffling
 - Slice layouts are not necessarily surjective anymore
- Solution2 swizzle during lowering of **AsyncCopy** to LLVM
 - We only swizzle **inside** warp boundaries => we **swap** elements between lanes of a warp
 - We can use **warp shuffling** to exchange global pointers
 - Use ds_bpermute to exchange global pointers and ballot to exchange the mask
 - If the fasted shared memory dim of the tensor is contiguous in HBM we can replace ds_bpermute by pointer arithmetic (BYPASS_PERMUTE)
- Solution 2 is currently implemented

Slicing

- If we slice AsyncCopy we need to be careful about the layouts
- Without slicing the rows of the logical tensor are contiguous in LDS
- If we slice this is no longer the case:
 - Assume threadsPerWarp = 16, dtype=float32
 - sizePerThread=[1,1], threadsPerWarp = [8,2]
 - Slice along dim1 should always be fine
 - 
 - sizePerThread=[1,1], threadsPerWarp = [4,4]
 - Slice along dim0
 - This look fine during lowering because we exhaust the fastest dim (shape [4,8])
 - Strided writes 

D	1	2	3	4	5	6	7	8
1	1	2	3	4	5	6	7	8
2	9	10	11	12	13	14	15	16
3	17	18	19	20	21	22	23	24
4	25	26	27	28	29	30	31	32
5	33	34	35	36	37	38	39	40
6	41	42	43	44	45	46	47	48
7	49	50	51	52	53	54	55	56
8	57	58	59	60	61	62	63	64

D	1	2	3	4	5	6	7	8
1	1	2	3	4	5	6	7	8
2	9	10	11	12	13	14	15	16
3	17	18	19	20	21	22	23	24
4	25	26	27	28	29	30	31	32
5	33	34	35	36	37	38	39	40
6	41	42	43	44	45	46	47	48
7	49	50	51	52	53	54	55	56
8	57	58	59	60	61	62	63	64

ThreadsPerWarp

- Assume threadsPerWarp = 16, dtype=float32
- sizePerThread=[1,1], threadsPerWarp = [8,2]
 - Slice along dim1 should always be fine
 - 

D	1	2	3	4	5	6	7	8
1	1	2	3	4	5	6	7	8
2	9	10	11	12	13	14	15	16
3	17	18	19	20	21	22	23	24
4	25	26	27	28	29	30	31	32
5	33	34	35	36	37	38	39	40
6	41	42	43	44	45	46	47	48
7	49	50	51	52	53	54	55	56
8	57	58	59	60	61	62	63	64

- sizePerThread=[1,1], threadsPerWarp = [4,4]
 - Slice along dim0
 - This look fine during lowering because we exhaust the fastest dim (shape [4,8])
 - Strided writes 

D	1	2	3	4	5	6	7	8
1	1	2	3	4	5	6	7	8
2	9	10	11	12	13	14	15	16
3	17	18	19	20	21	22	23	24
4	25	26	27	28	29	30	31	32
5	33	34	35	36	37	38	39	40
6	41	42	43	44	45	46	47	48
7	49	50	51	52	53	54	55	56
8	57	58	59	60	61	62	63	64

Open Problems

- Improve swizzling computations
 - Every additional instruction we get in the hotloop is * 4 for 300x
 - Experiment with different swizzle patterns
 - Try to make Solution1 work
- Include non async loads in vmcnt calculation
 - Important since scales might not be pipelined with AsyncCopy

